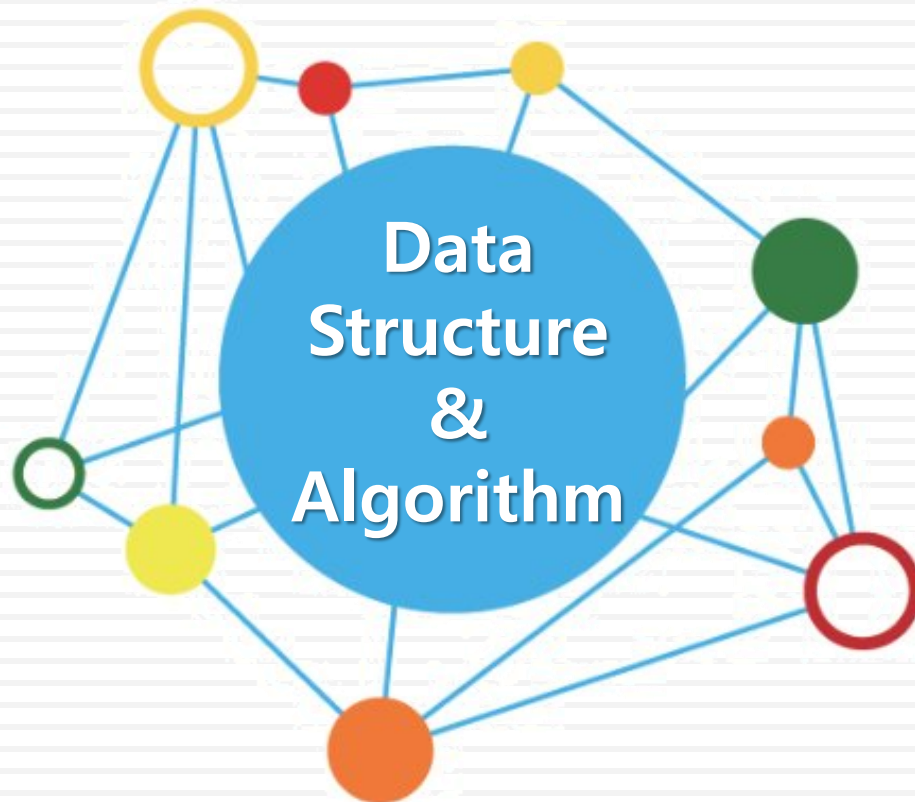


데이터 구조론



유길현

순차 데이터구조와
선형 리스트

❖ 순차 데이터 구조의 개념

- 구현할 자료들을 논리적 순서로 메모리에 연속 저장하는 방식
- 논리적인 순서와 물리적인 순서가 항상 일치해야 함
- C 프로그래밍에서 순차 데이터 구조의 구현 방식을 제공하는 프로그램 기법은 **배열**

구분	순차 자료구조	연결 자료구조
메모리 저장방식	메모리의 저장 시작 위치부터 빈 자리 없이 자료를 순서대로 연속하여 저장한다. 논리적인 순서와 물리적인 순서가 일치하는 구현 방식이다.	메모리에 저장된 물리적 위치나 물리적 순서에 상관없이 링크에 의해 논리적인 순서를 표현하는 구현 방식이다.
연산 특징	삽입 삭제 연산을 해도 빈자리 없이 자료가 순서대로 연속하여 저장된다. 변경된 논리적인 순서와 저장된 물리적인 순서가 일치한다.	삽입 삭제 연산을 하여 논리적인 순서가 변경되어도, 링크 정보만 변경되고 물리적 순서는 변경되지 않는다.
프로그램 기법	배열을 이용한 구현	포인터를 이용한 구현

❖ 리스트(List)

- 자료를 구조화하는 가장 기본적인 방법은 나열하는 것
- 리스트의 예

동창 이름	좋아하는 음식	오늘의 할 일
상원	김치찌개	운동
상범	닭 볶음탕	자료구조 스터디
수영	된장찌개	과제제출
현정	잡채	동아리 공연 연습
...	...	...

❖ 선형 리스트(Linear List)

- 순서 리스트
- 자료들 간에 순서를 갖는 리스트
- 선형 리스트의 예

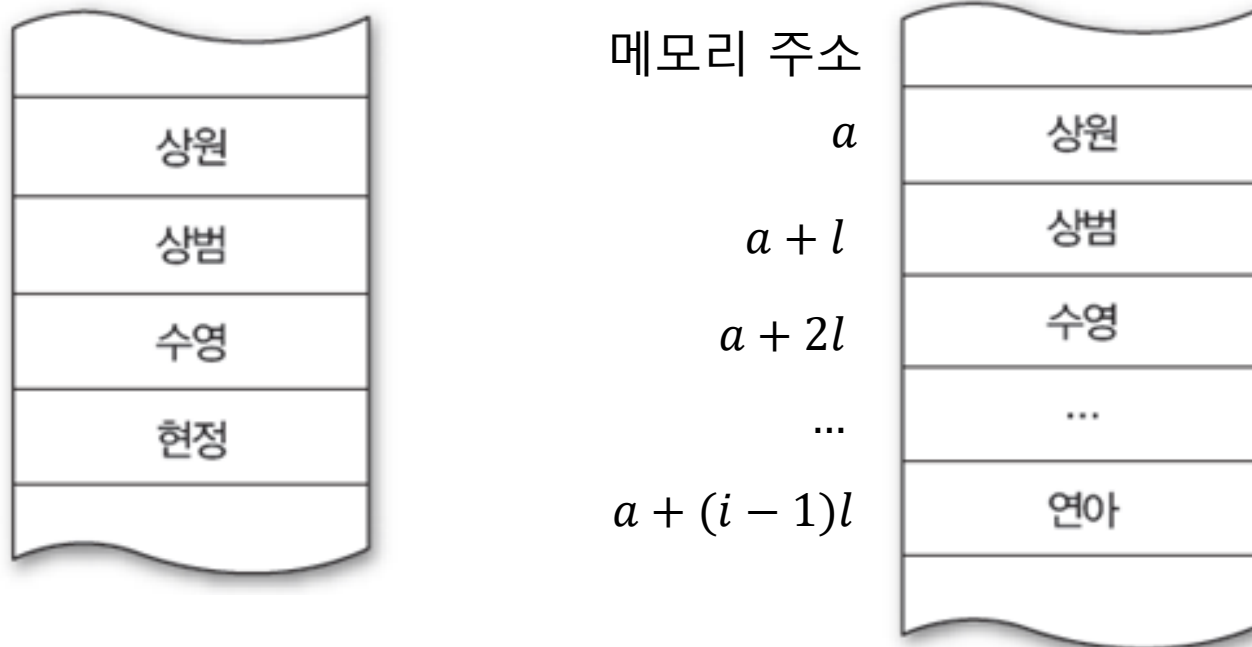
동창 이름		좋아하는 음식		오늘의 할 일	
1	상원	1	김치찌개	1	운동
2	상범	2	닭볶음탕	2	자료구조 스터디
3	수영	3	된장찌개	3	과제제출
4	현정	4	잡채	4	동아리 공연 연습
	...		...		...

❖ 선형 리스트(Linear List)

- 리스트 표현 형식
 - 리스트 이름 = (원소1, 원소2, ..., 원소 n)
- 선형 리스트에서 원소를 나열한 순서는 원소들의 순서가 된다.
- 동창이름 선형 리스트 표현
 - 동창 = (상원, 상범, 수영, 현정)
- 공백 리스트
 - 원소가 하나도 없는 리스트
 - 빈 괄호를 사용하여 표현
 - 공백리스트이름 = ()

❖ 선형 리스트의 저장

- 원소들의 논리적 순서와 같은 순서로 메모리에 저장
- 순차 자료구조
 - 원소들의 논리적 순서 = 원소들이 저장된 물리적 순서
 - 동창 선형 리스트가 메모리에 저장된 물리적 구조



❖ 선형 리스트의 저장

- 순차 자료구조의 원소 위치 계산
 - 선형 리스트가 저장된 시작 위치 : a
 - 원소의 길이 : l
 - i 번째 원소의 위치 = $a + (i - 1) \times l$

메모리 주소

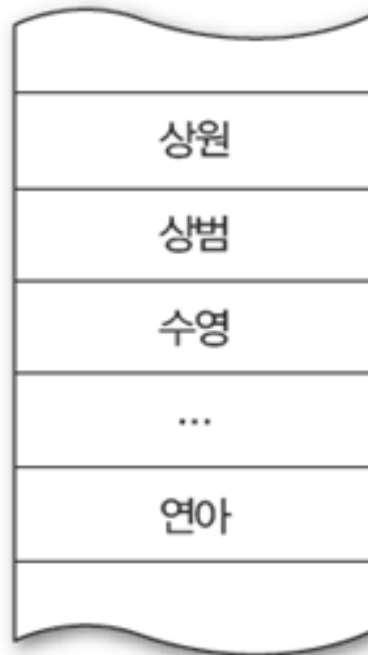
a

$a + l$

$a + 2l$

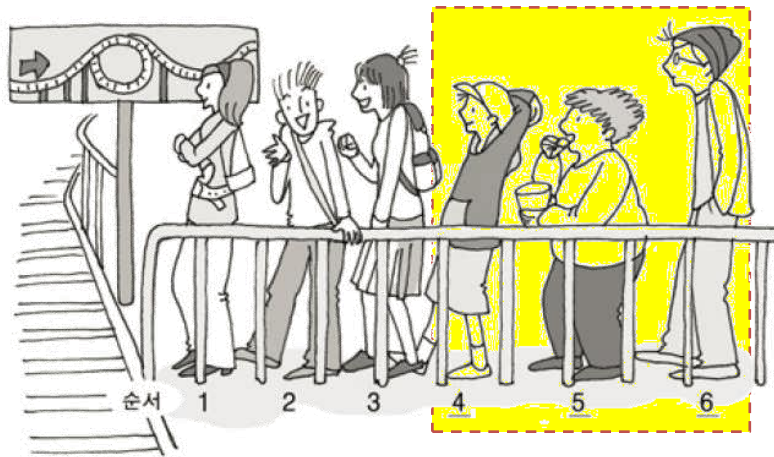
...

$a + (i - 1)l$

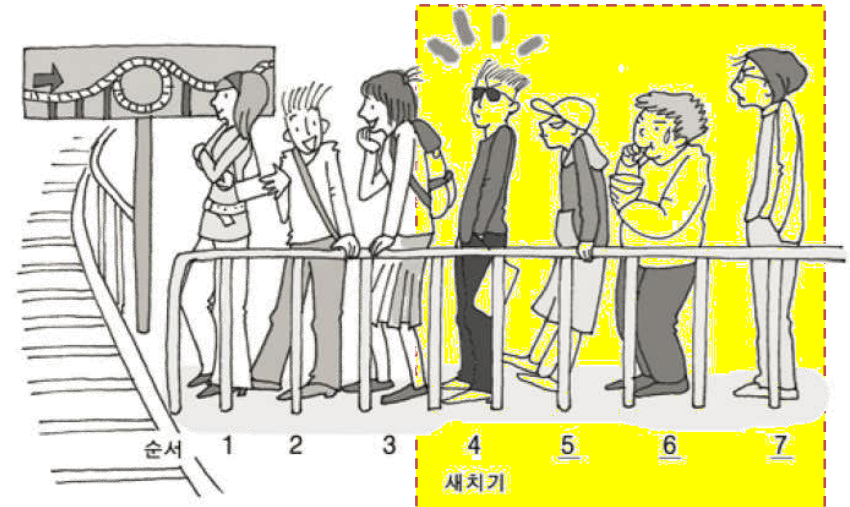


❖ 선형 리스트에서 원소 삽입

- 선형리스트 중간에 원소가 삽입되면, 그 이후의 원소들은 한 자리씩 자리를 뒤로 이동하여 물리적 순서를 논리적 순서와 일치 시킴



(a) 새치기 전



(b) 새치기 후

❖ 선형 리스트에서 원소 삽입

■ 원소 삽입 방법

① 원소를 삽입할 빈 자리 만들기

☞ 삽입할 자리 이후의 원소들을 한 자리씩 뒤로 자리 이동

② 준비한 빈 자리에 원소 삽입하기

(a) 원소 삽입 전

0	1	2	3	4	5	6
10	20	40	50	60	70	

(b) 원소 삽입 후

0	1	2	3	4	5	6
10	20		40	50	60	70
		자리 이동	자리 이동	자리 이동	자리 이동	
0	1	2	3	4	5	6
10	20	30	40	50	60	70
		↑				
		원소 30 삽입				

❖ 선형 리스트에서 원소 삽입

- 삽입할 자리를 만들기 위한 자리이동 횟수
 - $(n+1)$ 개의 원소로 이루어진 선형 리스트에서 k 번 자리에 원소를 삽입하는 경우 : k 번 원소부터 마지막 인덱스 n 번 원소까지 $(n-k+1)$ 개의 원소를 이동

$$\text{이동횟수} = n - k + 1$$

$$= \text{마지막 원소의 인덱스} - \text{삽입할 자리의 인덱스} + 1$$

❖ 선형 리스트에서 원소 삽입 프로그램 코드

```
#include <stdio.h>

int main()
{
    int LA[] = { 10, 20, 40, 50, 60, 70 };
    int item = 30, k = 2, n = 6;
    int i = 0, j = n;

    printf("원소 삽입전 배열 값 :\n");

    for (i = 0; i < n; i++)
    {
        printf("list[%d] = %d\n", i, LA[i]);
    }
}
```

❖ 선형 리스트에서 원소 삽입 프로그램 코드

```
n = n + 1;
while (j >= k)
{
    LA[j + 1] = LA[j];
    j = j - 1;
}
LA[k] = item;

printf("원소 삽입후 배열 값 :\n");

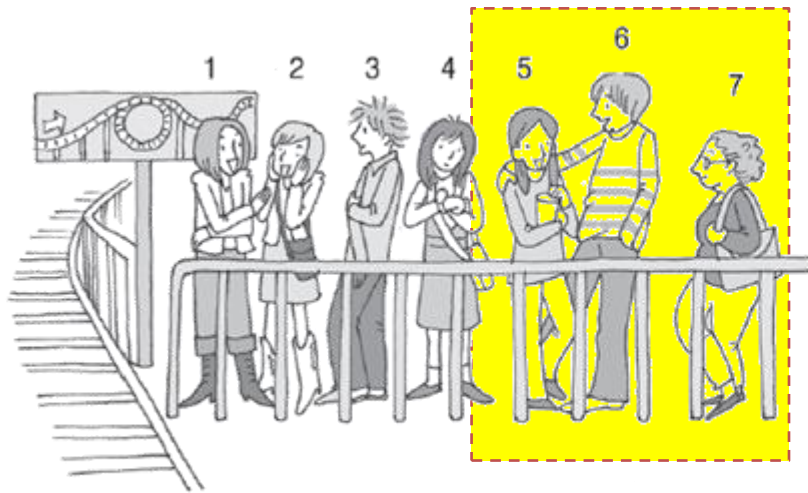
for (i = 0; i < n; i++)
{
    printf("LA[%d] = %d\n", i, LA[i]);
}

return 0;
}
```

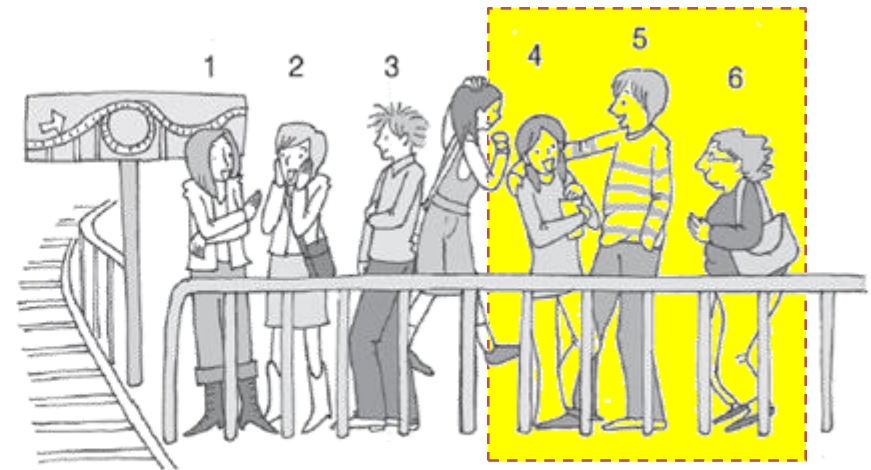
순차 자료구조와 선형 리스트의 이해

❖ 선형 리스트에서 원소 삭제

- 선형리스트 중간에서 원소가 삭제되면, 그 이후의 원소들은 한 자리씩 자리를 앞으로 이동하여 물리적 순서를 논리적 순서와 일치시킴



(a) 줄에서 나가기 전



(b) 줄에서 나간 후

❖ 선형 리스트에서 원소 삭제

■ 원소 삭제 방법

① 원소를 삭제하기

② 삭제한 빈 자리 채우기

☞ 삭제한 자리 이후의 원소들을 한자리씩 앞으로 자리 이동

(a) 원소 삭제 전

0	1	2	3	4	5	6
10	20	30	40	50	60	70

(b) 원소 삭제 후

0	1	2	3	4	5	6
10	20		40	50	60	70

원소 30 삭제

0	1	2	3	4	5	6
10	20	40	50	60	70	

자리 이동

자리 이동

자리 이동

자리 이동

❖ 선형 리스트에서 원소 삭제

- 삭제 후, 빈자리를 채우기 위한 자리이동 횟수
 - $(n+1)$ 개의 원소로 이루어진 선형 리스트에서 k 번 자리의 원소를 삭제한 경우 : $(k+1)$ 번 원소부터 마지막 n 번 원소까지 $(n-(k+1)+1)$ 개의 원소를 이동

$$\begin{aligned}\text{이동횟수} &= n-(k+1)+1 = n-k \\ &= \text{마지막 원소의 인덱스} - \text{삭제한 자리의 인덱스}\end{aligned}$$

❖ 선형 리스트에서 원소 삭제 프로그램 코드

```
#include <stdio.h>

int main()
{
    int LA[] = { 10, 20, 30, 40, 50, 60, 70 };
    int k = 3, n = 7;
    int i, j;

    printf("원소 삭제전 배열값 :\n");
    for (i = 0; i < n; i++)
    {
        printf("LA[%d] = %d \n", i, LA[i]);
    }
}
```


❖ 선형 리스트에서 원소 삭제 프로그램 코드

```
n = n - 1;
j = k;
while (j < n) {
    LA[j - 1] = LA[j];
    j = j + 1;
}

printf("원소 삭제후 배열값 : \n");
for (i = 0; i < n; i++)
{
    printf("LA[%d] = %d \n", i, LA[i]);
}

return 0;
}
```

❖ 선형 리스트의 구현

- 순차 구조의 배열을 사용
 - 배열 : <인덱스, 원소>의 순서쌍의 집합
 - 배열의 인덱스 : 배열 원소의 순서 표현

❖ 1차원 배열을 이용한 선형 리스트 구현

- 분기별 노트북 판매량 리스트

분기	1/4 분기	2/4 분기	3/4 분기	4/4 분기
판매량	157	209	251	312

선형 리스트의 구현

❖ 1차원 배열을 이용한 선형 리스트 구현

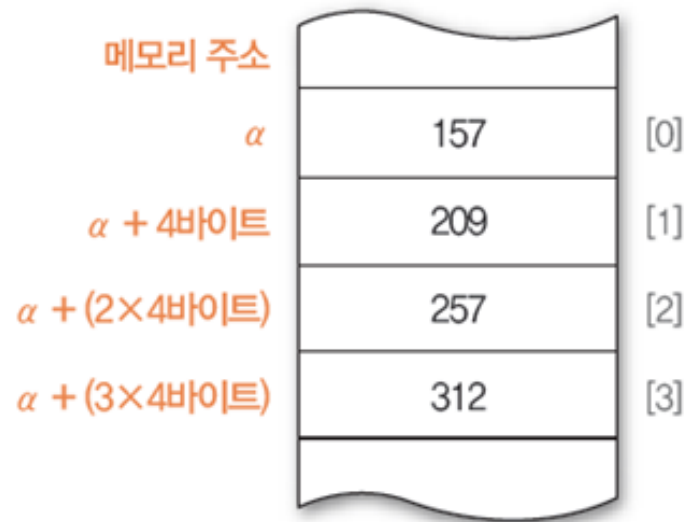
■ 1차원 배열을 이용한 구현

- `int sale[4] = { 157, 209, 251, 312};`

- 논리적 구조

	[0]	[1]	[2]	[3]
sale	157	209	251	312

- 물리적 구조



❖ 1차원 배열을 이용한 선형 리스트 구현

- 원소의 논리적 물리적 순서 확인 프로그램

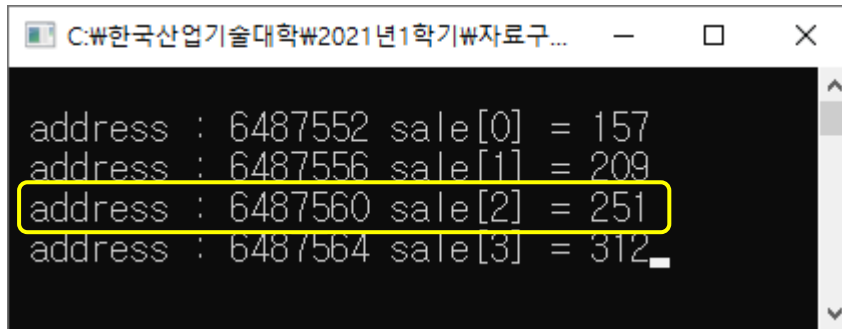
```
#include <stdio.h>

void main()
{
    int i, sale[4] = { 157, 209, 251, 312 };

    for (i = 0; i < 4; i++)
    {
        printf("\n address : %u sale[%d] = %d", &sale[i], i, sale[i]);
    }
    getchar();
}
```

❖ 1차원 배열을 이용한 선형 리스트 구현

■ 실행 결과



```
C:\₩한국산업기술대학₩2021년1학기₩자료구... - □ ×  
address : 6487552 sale[0] = 157  
address : 6487556 sale[1] = 209  
address : 6487560 sale[2] = 251  
address : 6487564 sale[3] = 312
```

■ 실행 결과 확인

- 배열 sale의 시작 주소 : 6487552
- $\text{sale}[2]=251$ 의 위치 = 시작 주소 + (인덱스 x 4바이트)
= $6487552 + (2 \times 4\text{바이트})$
= 6487560
- 논리적인 순서대로 메모리에 연속하여 저장된 순차 구조임을 확인!

선형 리스트의 구현

❖ 2차원 배열을 이용한 선형 리스트 구현

- 2016~2017년 분기별 노트북 판매량 리스트

년 \ 분기	1/4 분기	2/4 분기	3/4 분기	4/4 분기
2016년	63	84	140	130
2017년	157	209	251	312

- 2차원 배열을 이용한 구현
 - `int sale[2][4] = {{63, 84, 140, 130}, {157, 209, 251, 312}};`
 - 2016~2017년 분기별 판매량 선형 리스트 논리적 구조

	[0]	[1]	[2]	[3]
sale [0]	63	84	140	130
[1]	157	209	251	312

❖ 2차원 배열을 이용한 선형 리스트 구현

■ 2차원 배열의 물리적 저장 방법

- 2차원의 논리적 순서를 1차원의 물리적 순서로 변환하는 방법 사용
- 행 우선 순서 방법(row major order)

☞ 2차원 배열의 첫 번째 인덱스인 행 번호를 기준으로 사용하는 방법

☞ $\text{sale}[0][0]=63, \text{sale}[0][1]=84, \text{sale}[0][2]=140,$
 $\text{sale}[0][3]=130, \text{sale}[1][0]=157, \text{sale}[1][1]=209,$
 $\text{sale}[1][2]=251, \text{sale}[1][3]=312$

☞ 원소의 위치 계산 방법 : $\alpha + (i \times n_j + j) \times \ell$

- 행의 개수가 n_i 이고 열의 개수가 n_j 인 2차원 배열 $A[n_i][n_j]$ 의 시작주소가 α 이고
- 원소의 길이가 ℓ 일 때, i 행 j 열 원소 즉, $A[i][j]$ 의 위치

❖ 2차원 배열을 이용한 선형 리스트 구현

■ 2차원 배열의 물리적 저장 방법

- 열 우선 순서 방법(column major order)

☞ 2차원 배열의 마지막 인덱스인 열 번호를 기준으로 사용하는 방법

sale[0][0]=63, sale[1][0]=157,

sale[0][1]=84, sale[1][1]=209,

sale[0][2]=140, sale[1][2]=251,

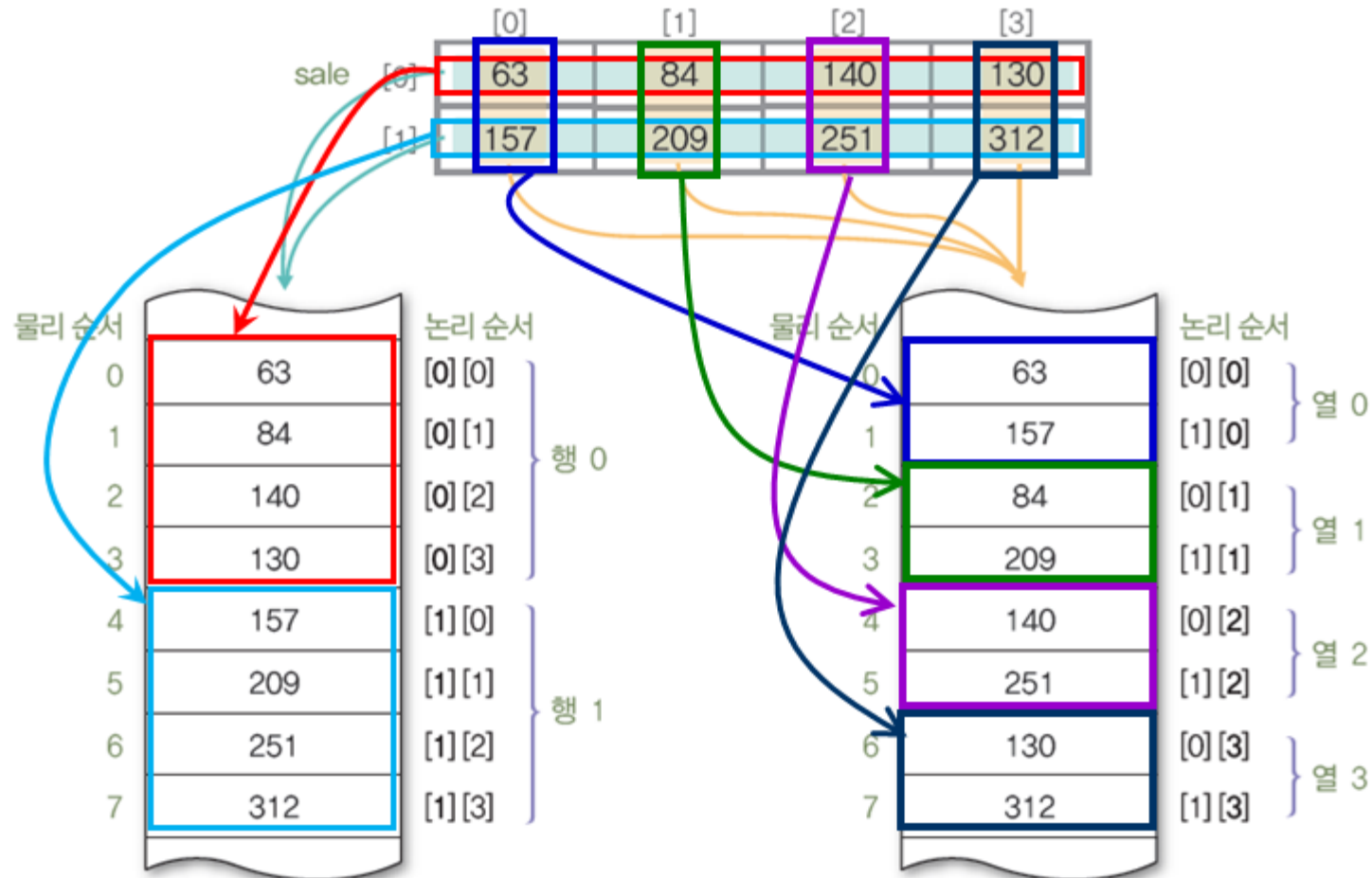
sale[0][3]=130, sale[1][3]=312

☞ 원소의 위치 계산 방법 : $\alpha + (j \times n_i + i) \times \ell$

선형 리스트의 구현

❖ 2차원 배열을 이용한 선형 리스트 구현

- 2차원 논리 순서를 1차원 물리 순서로 변환



행 우선 순서방법

열 우선 순서방법

❖ 2차원 배열을 이용한 선형 리스트 구현

- 2차원 배열의 논리적·물리적 순서 확인하기 프로그램 : [교재 131p](#)

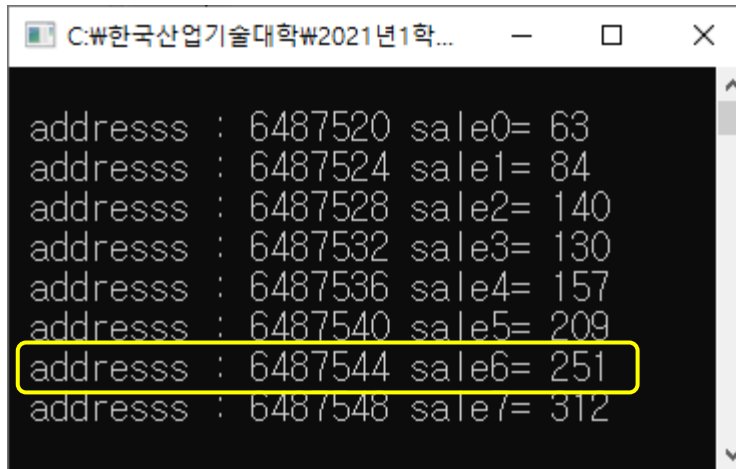
```
#include <stdio.h>

void main()
{
    int i, n = 0, *ptr;
    int sale[2][4] = {{63,84, 140, 130},
                     {157, 209, 251, 312}};    // 2차원 배열의 초기화

    ptr = &sale[0][0];
    for (i = 0; i < 8; i++)
    {
        printf("\n addresss : %u sale%d= %d", ptr, i, *ptr);
        ptr++;
    }
    getchar();
}
```

❖ 2차원 배열을 이용한 선형 리스트 구현

■ 실행 결과



```
C:\₩한국산업기술대학₩2021년1학...  
addressss : 6487520 sale0= 63  
addressss : 6487524 sale1= 84  
addressss : 6487528 sale2= 140  
addressss : 6487532 sale3= 130  
addressss : 6487536 sale4= 157  
addressss : 6487540 sale5= 209  
addressss : 6487544 sale6= 251  
addressss : 6487548 sale7= 312
```

■ 실행 결과 확인

- 시작 주소 $\alpha=6487520$, $n_i=2$, $n_j=4$, $i=1$, $j=1$, $\ell=4$
- $\text{sale}[1][2]=251$ 의 위치 $= \alpha + (i \times n_j + j) \times \ell$
 $= 6487520 + (1 \times 4 + 2) \times 4$
 $= 6487520 + 24$
 $= 6487544$
- C 컴파일러가 행 우선 순서 방법으로 2차원 배열을 저장함을 확인!

선형 리스트의 구현

❖ 3차원 배열을 이용한 선형 리스트의 구현

- 1팀과 2팀의 분기별 노트북 판매량

팀	분기		1/4 분기	2/4 분기	3/4 분기	4/4 분기
	년					
1 팀	2016년		63	84	140	130
	2017년		157	209	251	312
2 팀	2016년		59	80	130	135
	2017년		149	187	239	310

- 3차원 배열을 이용한 구현

```
int sale[2][4] = {{63, 84, 140, 130},  
                  {157, 209, 251, 312},  
                  {59, 80, 130, 135},  
                  {149, 187, 239, 310}};
```



❖ 3차원 배열을 이용한 선형 리스트의 구현

■ 3차원 배열의 물리적 저장 방법

- 3차원의 논리적 순서를 1차원의 물리적 순서로 변환하는 방법 사용
- 면 우선 순서 방법
 - ☞ 3차원 배열의 첫 번째 인덱스인 면 번호를 기준으로 사용하는 방법
 - ☞ 원소의 위치 계산 방법 : $\alpha + \{(i \times n_j \times n_k) + (j \times n_k) + k\} \times \ell$
면의 개수가 n_i , 행의 개수가 n_j , 열의 개수가 n_k 인 3차원 배열 $A[n_i][n_j][n_k]$, 시작주소가 α 이고 원소의 길이가 ℓ 일 때, i 면 j 행 k 열 원소 즉, $A[i][j][k]$ 의 위치
- 열 우선 순서 방법
 - ☞ 3차원 배열의 마지막 인덱스인 열 번호를 기준으로 사용하는 방법
 - ☞ 원소의 위치 계산 방법 : $\alpha + \{(k \times n_j \times n_i) + (j \times n_i) + i\} \times \ell$

❖ 3차원 배열을 이용한 선형 리스트의 구현

- 3차원의 논리 순서에 대한 1차원의 물리 순서 변환

물리 순서		논리 순서	
0	63	[0][0][0]	면0
1	84	[0][0][1]	
2	140	[0][0][2]	
3	130	[0][0][3]	
4	157	[0][1][0]	
5	209	[0][1][1]	
6	251	[0][1][2]	
7	312	[0][1][3]	
8	59	[1][0][0]	면1
9	80	[1][0][1]	
10	130	[1][0][2]	
11	135	[1][0][3]	
12	149	[1][1][0]	
13	187	[1][1][1]	
14	239	[1][1][2]	
15	310	[1][1][3]	

면 우선 순서 방법

물리 순서		논리 순서	
0	63	[0][0][0]	열0
1	59	[1][0][0]	
2	157	[0][1][0]	
3	149	[1][1][0]	
4	84	[0][0][1]	열1
5	80	[1][0][1]	
6	209	[0][1][1]	
7	187	[1][1][1]	
8	140	[0][0][2]	열2
9	130	[1][0][2]	
10	251	[0][1][2]	
11	239	[1][1][2]	
12	130	[0][0][3]	열3
13	135	[1][0][3]	
14	312	[0][1][3]	
15	310	[1][1][3]	

열 우선 순서 방법

❖ 3차원 배열을 이용한 선형 리스트의 구현

- 3차원 배열의 논리적·물리적 순서 확인하기 프로그램 : [교재 134p](#)

```
#include <stdio.h>

void main()
{
    int i, n = 0, *ptr;
    int sale[2][2][4] = {{{63, 84, 140, 130},          // 3차원 배열의 초기화
                          {157, 209, 251, 312}},
                          {{59, 80, 130, 135},
                          {149, 187, 239, 310}}};

    ptr = &sale[0][0][0];
    for (i = 0; i < 16; i++){
        printf("\n addresss : %u sale%2d= %3d", ptr, i, *ptr);
        ptr++;
    }
    getchar();
}
```

❖ 3차원 배열을 이용한 선형 리스트의 구현

■ 실행 결과

```
C:\₩한국산업기술대학₩2021년1학기₩데...
addresss : 6487488 sale 0= 63
addresss : 6487492 sale 1= 84
addresss : 6487496 sale 2= 140
addresss : 6487500 sale 3= 130
addresss : 6487504 sale 4= 157
addresss : 6487508 sale 5= 209
addresss : 6487512 sale 6= 251
addresss : 6487516 sale 7= 312
addresss : 6487520 sale 8= 59
addresss : 6487524 sale 9= 80
addresss : 6487528 sale10= 130
addresss : 6487532 sale11= 135
addresss : 6487536 sale12= 149
addresss : 6487540 sale13= 187
addresss : 6487544 sale14= 239
addresss : 6487548 sale15= 310
```

- 시작 주소 $\alpha=6487488$, $n_i=2$, $n_j=2$, $n_k=4$, $i=1$, $j=1$, $k=2$, $\ell=4$
- $\text{sale}[1][1][2]=239$ 의 위치
$$= \alpha + \{(i \times n_j \times n_k) + (j \times n_k) + k\} \times \ell$$
$$= 6487488$$
$$+ \{(1 \times 2 \times 4) + (1 \times 4) + 2\} \times 4$$
$$= 6487488 + 56 = 6487544$$
- C 컴파일러가 면 우선 순서 방법으로 3차원 배열을 저장함을 확인!

❖ 다항식의 선형 리스트 표현

■ 다항식의 개념

aX^e 형식의 항들의 합으로 구성된 식

a : 계수(coefficient)

X : 변수(variable)

e : 지수(exponent)

■ 다항식의 특징

- 지수에 따라 내림차순으로 항을 나열
- 다항식의 차수 : 가장 큰 지수
- 다항식 항의 최대 개수 = (차수 + 1)개

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x^1 + a_0 x^0$$

❖ 다항식의 추상 데이터형(1/2)

ADT Polynomial

데이터 : 지수(e_i)-계수(a_i)의 순서쌍 $\langle e_i, a_i \rangle$ 의 집합으로 표현된
다항식 $p(x) = a_0x^{e_0} + a_1x^{e_1} + \dots + a_nx^{e_n}$. (e_i 는 음이 아닌 정수)

// p, p_1, p_2 는 다항식이고, a 는 계수, e 는 지수를 나타낸다.

연산 : $p, p_1, p_2 \in \text{Polynomial}$; $a \in \text{Coefficient}$; $e \in \text{Exponent}$;

// 다항식 $p(x)=0$ 을 만드는 연산

zeroP() ::= **return** polynomial $p(x)=0$;

// 다항식 p 가 0인지 검사하여 0이면 true를 반환하는 연산

isZeroP(p) ::= **if** (p) **then false**;
 else return true;

// 다항식 p 에서 지수가 e 인 항의 계수 a 를 구하는 연산

coef(p,e) ::= **if** ($\langle e, a \rangle \in p$) **then return** a ;
 else return 0;

// 다항식 p 에서 최대 지수를 구하는 연산

maxExp(p) ::= **return** $\max(p.\text{Exponent})$;

❖ 다항식의 추상 데이터형(2/2)

```
// 다항식p에 지수가 e인 항이 없는 경우에 새로운 항 <e, a>를 추가하는 연산  
addTerm(p,a,e) ::= if ( $e \in p.\text{Exponent}$ ) then return error;  
                          else return p after inserting the term <e,a>;
```

```
// 다항식p에서 지수가 e인 항 <e,a>를 삭제하는 연산  
delTerm(p,e) ::= if ( $e \in p.\text{Exponent}$ ) then return p after removing the term  
                          <e,a>;  
                          else return error;
```

```
// 다항식p의 모든 항에 ax항을 곱하는 연산  
multTerm(p,a,e) ::= return ( $p * ax$ );
```

```
// 두 다항식p1과 p2의 합을 구하는 연산  
addPoly(p1,p2) ::= return ( $p1 + p2$ );
```

```
// 두 다항식p1과 p2의 곱을 구하는 연산  
multPoly(p1,p2) ::= return ( $p1 * p2$ );
```

End Polynomial

다항식의 순차 데이터구조 구현

❖ 1차원 배열을 이용한 순차 데이터구조 표현

- 차수가 n 인 다항식을 $(n+1)$ 개의 원소를 가지는 1차원 배열로 표현
 - 배열 인덱스 i : 지수($n-i$)을 의미
 - 배열 인덱스 i 의 원소 : 지수($n-i$)항의 계수
 - 다항식에 포함되지 않은 지수의 항에 대한 원소에 0 저장
- $$P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x^1 + a_0 x^0$$
- n 차 다항식 $P(x)$ 의 순차 자료구조 표현

인덱스	[0]	[1]	...	[n-1]	[n]
P	a_n	a_{n-1}	...	a_1	a_0

$A(x) = 4x^3 + 3x^2 + 2$

	[0]	[1]	[2]	[3]
A	4	3	0	2

❖ 1차원 배열을 이용한 순차 데이터구조 표현

- 희소 다항식에 대한 1차원 배열 저장
 - 예) $B(x) = 3x^{1000} + x + 4$

	[0]	[1]	[2]	[3]	...	[997]	[998]	[999]	[1000]
B	3	0	0	0	...	0	0	1	4

- 차수가 1000이므로 크기가 1001인 배열을 사용하는데, 항이 3개 뿐이므로 배열 원소 중에서 3개만 사용 하고 998개의 나머지 배열 원소 공간이 낭비된다.

❖ 2차원 배열을 이용한 순차 데이터구조 표현

- 다항식의 각 항에 대한 <지수, 계수>의 쌍을 2차원 배열에 저장
 - 2차원 배열의 행의 개수 : 다항식의 항의 계수
 - 2차원 배열의 열의 개수 : 2
 - 예) $B(x)=3x^{1000} + x + 4$ 의 2차원 배열 표현

	[0]	[1]	
[0]	1000	3	→ $3x^{1000}$
[1]	1	1	→ x
[2]	0	4	→ 4

❖ 다항식의 덧셈 알고리즘 : $A(x) + B(x) = C(x)$

addPoly(A, B)

// 주어진 두 다항식 A와 B를 더하여 결과 다항식 C를 반환하는 알고리즘

$C \leftarrow \text{zeroP}();$

while (**not** isZeroP(A) **and not** isZeroP(B)) **do** {

case {

 maxExp(A) < maxExp(B) :

$C \leftarrow \text{addTerm}(C, \text{coef}(B, \text{maxExp}(B)), \text{maxExp}(B));$

$B \leftarrow \text{delTerm}(B, \text{maxExp}(B));$

 maxExp(A) = maxExp(B) :

$\text{sum} \leftarrow \text{coef}(A, \text{maxExp}(A)) + \text{coef}(B, \text{maxExp}(B));$

if (sum ≠ 0) **then** $C \leftarrow \text{addTerm}(C, \text{sum}, \text{maxExp}(A));$

$A \leftarrow \text{delTerm}(A, \text{maxExp}(A));$

$B \leftarrow \text{delTerm}(B, \text{maxExp}(B));$

 maxExp(A) > maxExp(B) :

$C \leftarrow \text{addTerm}(C, \text{coef}(A, \text{maxExp}(A)), \text{maxExp}(A));$

$A \leftarrow \text{delTerm}(A, \text{maxExp}(A));$

 }

}

if (**not** isZeroP(A)) **then** A의 나머지 항들을 C에 복사;

else if (**not** isZeroP(B)) **then** B의 나머지 항들을 C에 복사;

return C;

End addPoly()

❖ 다항식의 덧셈 프로그램(1/4) : [교재 140p](#) 예제 3-4

```
#include <stdio.h>
#define MAX(a, b) ((a>b) ? a : b)
#define MAX_DEGREE 50

typedef struct {
    int degree;
    float coef[MAX_DEGREE];
} polynomial;

polynomial addPoly(polynomial A, polynomial B)
{
    polynomial C;
    int A_index = 0, B_index = 0, C_index = 0;
    int A_degree = A.degree, B_degree = B.degree;
    C.degree = MAX(A.degree, B.degree);
```


❖ 다항식의 덧셈 프로그램(2/4) : [교재 140p](#) 예제 3-4

```
while (A_index <= A.degree && B_index <= B.degree) {  
    if (A_degree > B_degree) {  
        C.coef[C_index++] = A.coef[A_index++];  
        A_degree--;  
    }  
    else if (A_degree == B_degree) {  
        C.coef[C_index++] = A.coef[A_index++] + B.coef[B_index++];  
        A_degree--;  
        B_degree--;  
    }  
    else {  
        C.coef[C_index++] = B.coef[B_index++];  
        B_degree--;  
    }  
}  
return C;  
}
```

❖ 다항식의 덧셈 프로그램(3/4) : [교재 140p](#) 예제 3-4

```
void printPoly(polynomial P)
{
    int i, degree;
    degree = P.degree;

    for (i = 0; i <= P.degree; i++){
        printf("%3.0fx^%d", P.coef[i], degree--);
        if (i < P.degree)
            printf("+");
    }
    printf("\n");
}
```

❖ 다항식의 덧셈 프로그램(4/4) : [교재 140p](#) 예제 3-4

```
void main()
{
    polynomial A = { 3, { 4, 3, 5, 0 } };
    polynomial B = { 4, { 3, 1, 0, 2, 1 } };

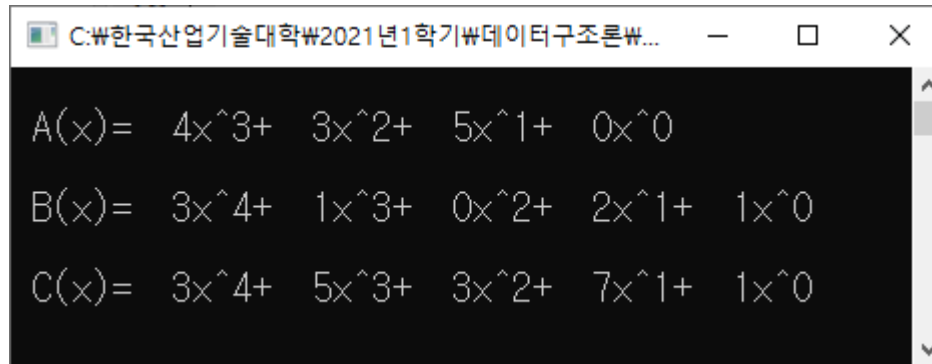
    printf("\n A(x)="); printPoly(A);
    printf("\n B(x)="); printPoly(B);

    polynomial C;
    C = addPoly(A, B);
    printf("\n C(x)="); printPoly(C);

    getchar();
}
```

❖ 다항식의 덧셈 프로그램

■ 실행 결과



A screenshot of a Windows command prompt window. The title bar shows the file path: C:\₩한국산업기술대학₩2021년1학기₩데이터구조론₩... The window contains three lines of text representing polynomial functions:

```
A(x)= 4x^3+ 3x^2+ 5x^1+ 0x^0  
B(x)= 3x^4+ 1x^3+ 0x^2+ 2x^1+ 1x^0  
C(x)= 3x^4+ 5x^3+ 3x^2+ 7x^1+ 1x^0
```

❖ 행렬의 선형 리스트 표현

- 행렬(matrix)의 개념
 - 행과 열로 구성된 자료구조
 - $m \times n$ 행렬 : 행 개수가 m 개, 열 개수가 n 개인 행렬
 - 정방행렬 : 행렬 중에서 m 과 n 이 같은 행렬
 - 전치행렬 : 행렬의 행과 열을 서로 바꿔 구성한 행렬

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \xrightarrow{\text{전치 행렬로 변환}} A' = \begin{bmatrix} a_{11} & a_{21} & \cdots & a_{m1} \\ a_{12} & a_{22} & \cdots & a_{m2} \\ a_{1n} & a_{2n} & \cdots & a_{mn} \end{bmatrix}$$

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} \xrightarrow{\text{전치행렬로 변환}} A' = \begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix}$$

행렬의 순차 데이터구조 구현

❖ 행렬의 자료구조 표현

■ 2차원 배열 사용

- $m \times n$ 행렬을 m 행 n 열의 2차원 배열로 표현

$A = \begin{vmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{vmatrix}$ 

$A[3][4]$

	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

$B = \begin{vmatrix} 0 & 0 & 2 & 0 & 0 & 0 & 12 \\ 0 & 0 & 0 & 0 & 7 & 0 & 0 \\ 23 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 31 & 0 & 0 & 0 \\ 0 & 14 & 0 & 0 & 0 & 25 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 6 \\ 52 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 11 & 0 & 0 \end{vmatrix}$ 

$B[8][7]$

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	0	0	2	0	0	0	12
[1]	0	0	0	0	7	0	0
[2]	23	0	0	0	0	0	0
[3]	0	0	0	31	0	0	0
[4]	0	14	0	0	0	25	0
[5]	0	0	0	0	0	0	6
[6]	52	0	0	0	0	0	0
[7]	0	0	0	0	11	0	0

❖ 행렬의 자료구조 표현

- 희소 행렬에 대한 2차원 배열 표현
 - 희소 행렬 B는 배열의 원소 56개 중 실제 사용하는 것은 0이 아닌 원소를 저장하는 10개뿐이므로 46개의 메모리 공간 낭비
 - 희소 행렬의 경우에는 0이 아닌 원소만 추출하여 <행 번호, 열 번호, 원소> 쌍으로 배열에 저장

B [8][7]

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	
[0]	0	0	2	0	0	0	12	<0, 2, 2> <0, 6, 12>
[1]	0	0	0	0	7	0	0	<1, 4, 7>
[2]	23	0	0	0	0	0	0	<2, 0, 23>
[3]	0	0	0	31	0	0	0	<3, 3, 31>
[4]	0	14	0	0	0	25	0	<4, 1, 14> <4, 5, 25>
[5]	0	0	0	0	0	0	6	<5, 6, 6>
[6]	52	0	0	0	0	0	0	<6, 0, 52>
[7]	0	0	0	0	11	0	0	<7, 4, 11>

❖ 행렬의 자료구조 표현

- 희소 행렬에 대한 2차원 배열 표현
 - 추출한 순서쌍을 2차원 배열의 행으로 저장
 - 원래의 행렬에 대한 정보를 순서쌍으로 작성하여 0번 행에 저장

(전체 행의 개수), (전체 열의 개수), (0이 아닌 원소의 개수)

⟨0, 2, 2⟩
⟨0, 6, 12⟩
⟨1, 4, 7⟩
⟨2, 0, 23⟩
⟨3, 3, 31⟩
⟨4, 1, 14⟩
⟨4, 5, 25⟩
⟨5, 6, 6⟩
⟨6, 0, 52⟩
⟨7, 4, 11⟩

2차원
배열에 저장

	[0]	[1]	[2]
[0]	8	7	10
[1]	0	2	2
[2]	0	6	12
[3]	1	4	7
[4]	2	0	23
[5]	3	3	31
[6]	4	1	14
[7]	4	5	25
[8]	5	6	6
[9]	6	0	52
[10]	7	4	11

❖ 희소 행렬의 추상 자료형

ADT Sparse_Matrix

데이터 : 3원소쌍 <행 인덱스, 열 인덱스, 원소 값>의 집합

// a, b는 희소행렬, c는 행렬, u, v는 행렬의 원소 값, i와 j는 행 인덱스와 열 인덱스

연산 : $a, b \in \text{Sparse_Matrix}$; $c \in \text{Matrix}$; $u, v \in \text{value}$; $i \in \text{Row}$; $j \in \text{Column}$;

// $m \times n$ 의 공백 희소행렬을 만드는 연산

smCreate(m,n) ::= return an empty sparse matrix with $m \times n$;

// 희소행렬 a[i,j]=v를 c[i,j]=v로 전치시킨 전치행렬 c를 구하는 연산

smTranspose(a) ::= return c where $c[i,j]=v$ when $a[i,j]=v$;

// 차수가 같은 희소행렬 a와 b를 합한 행렬 c를 구하는 연산

smAdd(a,b) ::= if (a.dimension = b.dimension)

then return c where $c[i,j]=v+u$ where $a[i,j]=v$ and $b[i,j]=u$;

else return error;

// 희소행렬 a의 열의 개수(n)와 희소행렬 b의 행의 개수(m)가 같은 경우에 두 행렬의 곱을 구하는 연산

smMulti(a,b) ::= if (a.n = b.m) **then return** c where $c[i,j]=a[i,k] \times b[k,j]$;

else return error;

End Sparse_Matrix

❖ 희소 행렬의 전치 연산 알고리즘

smTranspose(a[])

```
m ← a[0,0]; // 희소 행렬 a의 행 수
n ← a[0,1]; // 희소 행렬 a의 열 수
v ← a[0,2]; // 희소 행렬 a에서 0이 아닌 원소 수
b[0,0] ← n; // 전치 행렬 b의 행 수 지정
b[0,1] ← m; // 전치 행렬 b의 열 수 지정
b[0,2] ← t; // 전치 행렬 b의 원소 수 지정
if (v > 0) then { // 0이 아닌 원소가 있는 경우에만 전치 연산 수행
    p ← 1;
    for (i←0; i < n; i←i+1) do { // 희소 행렬 a의 열 별로 전치 반복 수행
        for (j←1; j ≤ v; j←j+1) do { // 0이 아닌 원소 수에 대해서만 반복 수행
            if (a[j,1]←i) then { // 현재의 열에 속하는 원소가 있으면 b[]에 삽입
                b[p,0] ← a[j,1];
                b[p,1] ← a[j,0];
                b[p,2] ← a[j,2];
                p ← p+1;
            }
        }
    }
}
return b[];
End smTranspose()
```

❖ 희소 행렬 만들기(1/5)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct{
    int row;
    int col;
    int value;
} SprsMtrx;
```

❖ 희소 행렬 만들기(2/5)

```
SprsMtrx *smCreate(int A[][7], int row, int col, int *cnt)
{
    int i, j;
    SprsMtrx *S;
    for (j = 0; j < row; j++)
    {
        for (i = 0; i < col; i++)
        {
            if (A[j][i])           // 0이 아닌 데이터 개수 구하기;
                (*cnt)++;
        }
    }
    // 초기값 때문에 공간이 1개 더 필요함
    S = (SprsMtrx*)malloc(sizeof(SprsMtrx)*((*cnt) + 1));

    // 시작 값 행, 열, 데이터 개수 넣기
    S[0].row = row;
    S[0].col = col;
    S[0].value = *cnt;
```

❖ 희소 행렬 만들기(3/5)

```
// 희소행렬의 각 원소에 값 할당하기
*cnt = 1;
for (j = 0; j < row; j++)
{
    for (i = 0; i < col; i++)
    {
        if (A[j][i])
        {
            S[*cnt].row = j;
            S[*cnt].col = i;
            S[*cnt++].value = A[j][i];
        }
    }
}
return S;
}
```

❖ 희소 행렬 만들기(4/5)

```
int main()
{
    int i, j, cnt = 0;

    // 변경 전 행렬 값
    int A[][7] = { {0, 0, 2, 0, 0, 0, 12},
                   {0, 0, 0, 0, 7, 0, 0},
                   {23, 0, 0, 0, 0, 0, 0},
                   {0, 0, 0, 31, 0, 0, 0},
                   {0, 14, 0, 0, 0, 25, 0},
                   {0, 0, 0, 0, 0, 0, 6},
                   {52, 0, 0, 0, 0, 0, 0},
                   {0, 0, 0, 0, 11, 0, 0} };
```

❖ 희소 행렬 만들기(5/5)

```
for (j = 0; j < 8; j++)
{
    for (i = 0; i < 7; i++)
    {
        printf("%4d", A[j][i]);
    }
    printf("\n");
}

SprsMtrx *S = smCreate(A, sizeof(A) / (sizeof(int)*7), 7, &cnt);

printf("=====희소 행렬 출력=====\n");    // 출력 하기
for (i = 0; i < cnt; i++)
{
    printf("%4d %4d %4d\n", S[i].row, S[i].col, S[i].value);
}
return 0;
}
```

❖ 입력된 숫자를 사용하여 가장 큰 수 만들기

각 자리의 숫자 0 부터 9로만 이루어진 문자열 주어졌을 때, 왼쪽에서 오른쪽으로 숫자를 확인하며 숫자 사이에 'x' 혹은 '+' 연산자를 넣어 결과적으로 만들어 질 수 있는 가장 큰 수를 구하는 프로그램을 작성 하세요 단 '+' 보다 'x'를 먼저 계산하는 일반적인 방식과는 달리 모든 연산은 왼쪽에서 부터 순서대로 이루어진다고 가정 합니다.

예를 들어 02984라는 문자열이 주어지면, 만들 수 있는 가장 큰 수는 $((0+2) \times 9) \times 8) \times 4=576$ 입니다.

- 입력 조건 : 첫째 줄에 여러 개의 숫자로 구성된 하나의 문자열이 주어 집니다. ($1 \leq$ 문자열의 길이 ≤ 20)
- 출력 조건 : 첫째 줄에 만들어 질 수 있는 가장 큰 수를 출력 합니다.

입력 예시 1

02984

출력 예시 1

576

입력 예시 2

567

출력 예시 2

210